

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 5”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu :	Dr. Wahyudi, S.T, M.T.
Asisten Praktikum :	Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum Praktikum Struktur Data untuk pertemuan pekan ke-5 ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk mendokumentasikan hasil praktikum mengenai struktur data Single Linked List (SLL). Fokus utama pada pekan ini adalah memahami konsep dasar senarai berantai tunggal, pembentukan struktur node, serta operasi-operasi fundamental seperti penambahan (insertion), penghapusan (deletion), dan pencarian (searching) data dalam senarai.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan demi perbaikan di masa mendatang.

Padang, 6 Mei, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	3
2.1 Class NodeSLL_2511532014	3
2.2 Class TambahSLL_2511532014.....	3
2.3 Class HapusSLL_2511532014	6
2.4 Class PencarianSLL_2511532014.....	9
BAB III KESIMPULAN	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Single Linked List (SLL) adalah sekumpulan simpul (node) yang disusun secara berurutan, di mana setiap simpul memiliki dua bagian utama: bagian data dan bagian penunjuk (pointer) yang merujuk ke simpul berikutnya dalam urutan tersebut. Berbeda dengan array yang memiliki ukuran statis dan penyimpanan memori yang berdekatan, SLL memungkinkan alokasi memori secara dinamis, sehingga sangat fleksibel dalam penambahan dan pengurangan data tanpa perlu menggeser elemen lainnya.

Pada praktikum pekan ke-5 ini, dilakukan pengujian mengenai cara membangun struktur node, melakukan operasi penelusuran (traversal), serta memanipulasi isi senarai baik di posisi depan, belakang, maupun posisi tertentu. Pemahaman terhadap mekanisme pointer sangat krusial dalam struktur ini agar hubungan antar simpul tidak terputus saat proses modifikasi dilakukan.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami struktur dasar simpul (*node*) pada Single Linked List.
2. Mampu mengimplementasikan operasi penambahan data di awal, akhir, dan posisi tengah senarai.
3. Mampu mengimplementasikan operasi penghapusan data pada *head*, *tail*, dan posisi tertentu.
4. Memahami mekanisme pencarian data (*searching*) dan penelusuran (*traversal*) pada senarai berantai.
5. Melatih logika penggunaan *pointer* untuk menghubungkan simpul-simpul dalam memori dinamis.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Efisiensi Memori: Mahasiswa memahami cara mengelola data yang ukurannya dapat berubah-ubah secara dinamis selama program berjalan.

2. Keterampilan Logika Pointer: Mengasah kemampuan dalam memanipulasi referensi objek dalam bahasa pemrograman Java.
3. Optimasi Struktur Data: Mahasiswa dapat menentukan kapan harus menggunakan Linked List dibandingkan Array berdasarkan kebutuhan operasi (seperti efisiensi penyisipan data).
4. Dasar Struktur Kompleks: Menjadi landasan untuk mempelajari struktur data yang lebih maju seperti Double Linked List, Circular Linked List, Trees, dan Graphs.

BAB II PEMBAHASAN

2.1 Class NodeSLL_2511532014

2.1.1 Langkah Pengerjaan

1. Mendeklarasikan variabel data_2014 bertipe integer untuk menyimpan nilai simpul.
2. Mendeklarasikan variabel next_2014 bertipe NodeSLL_2511532014 sebagai pointer ke simpul berikutnya.
3. Membuat konstruktor untuk menginisialisasi simpul dengan nilai tertentu dan mengatur next_2014 ke null.

2.1.2 Kode Program

```
1 package pekan5_2511532014;
2
3 public class NodeSLL_2511532014 {
4     //node bagian data
5     int data_2014;
6     //pointer ke node berikutnya
7     NodeSLL_2511532014 next_2014;
8     //konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511532014 (int data_2014) {
10         this.data_2014 = data_2014;
11         this.next_2014 = null;
12     }
13
14 }
15
```

2.1.3 Analisis

Kelas ini adalah fondasi dari seluruh operasi SLL. Setiap simpul dibentuk sebagai objek mandiri yang membawa muatan data dan alamat memori simpul tetangganya. Pengaturan awal next_2014 = null sangat penting untuk menandai akhir dari sebuah simpul sebelum disambungkan ke simpul lain.

2.2 Class TambahSLL_2511532014

2.2.1 Langkah Pengerjaan

1. Membuat metode insertAtFront untuk menambahkan simpul baru di posisi head.
2. Membuat metode insertAtEnd untuk menambahkan simpul di posisi akhir dengan cara melakukan penelusuran hingga menemukan simpul yang next-nya null.

3. Membuat metode insertPos untuk menyisipkan data pada posisi spesifik dengan mengatur ulang rantai pointer pada simpul sebelumnya dan simpul baru.

2.2.2 Kode Program

```
1 package pekan5_2511532014;
2
3 public class TambahSLL_2511532014 {
4     public static NodeSLL_2511532014 insertAtFront_2511532014
5     (NodeSLL_2511532014 head_2014, int value_2014) {
6         NodeSLL_2511532014 new_node_2014 = new
7         NodeSLL_2511532014(value_2014);
8         new_node_2014.next_2014 = head_2014;
9         return new_node_2014;
10    }
11    //fungsi menambahkan node diakhir SLL
12    public static NodeSLL_2511532014 insertAtEnd_2511532014
13    (NodeSLL_2511532014 head_2014, int value_2014) {
14        //buat sebuah node dengan sebuah nilai
15        NodeSLL_2511532014 newNode_2014 = new
16        NodeSLL_2511532014(value_2014);
17        //jika list kosong maka node akan jadi head
18        if (head_2014 == null) {
19            return newNode_2014;
20        }
21        //simpan head ke variabel sementara
22        NodeSLL_2511532014 last_2014 = head_2014;
23        //telusuri ke node akhir
24        while (last_2014.next_2014 != null) {
25            last_2014 = last_2014.next_2014;
26        }
27        //ubah pointer
28        last_2014.next_2014 = newNode_2014;
29        return head_2014;
30    }
31    static NodeSLL_2511532014 GetNode_2511532014 (int data_2014) {
32        return new NodeSLL_2511532014(data_2014);
33    }
34
35    static NodeSLL_2511532014
36    insertPos_2511532014(NodeSLL_2511532014 headNode_2014, int
37    position_2014, int value_2014) {
38        NodeSLL_2511532014 head_2014 = headNode_2014;
39        if (position_2014 < 1) {
40            System.out.print("Invalid position");
41        }
42        if (position_2014 == 1) {
43            NodeSLL_2511532014 new_node_2014 = new
44            NodeSLL_2511532014(value_2014);
45            new_node_2014.next_2014 = head_2014;
46            return new_node_2014;
47        } else {
48            while (position_2014-- != 0) {
49                if (position_2014 == 1) {
50                    NodeSLL_2511532014 newNode_2014 =
51                    GetNode_2511532014(value_2014);
```

```

44         newNode_2014.next_2014 =
headNode_2014.next_2014;
45         headNode_2014.next_2014 =
newNode_2014;
46         break;
47     }
48     headNode_2014 = headNode_2014.next_2014;
49     }
50     if (position_2014 != 1)
51         System.out.print("Posisi diluar
jangkauan");
52     }
53     return head_2014;
54 }
55 public static void printList_2511532014(NodeSLL_2511532014
head_2014) {
56     NodeSLL_2511532014 curr_2014 = head_2014;
57     while (curr_2014.next_2014 != null) {
58         System.out.print(curr_2014.data_2014 + "-->");
59         curr_2014 = curr_2014.next_2014;
60     }
61     if (curr_2014.next_2014 == null) {
62         System.out.print(curr_2014.data_2014);
63     }
64     System.out.println();
65 }
66
67 public static void main(String[] args) {
68     //buat linked list 2-->3-->5-->6
69     NodeSLL_2511532014 head_2014 = new NodeSLL_2511532014(2);
70     head_2014.next_2014 = new NodeSLL_2511532014(3);
71     head_2014.next_2014.next_2014 = new
NodeSLL_2511532014(5);
72     head_2014.next_2014.next_2014.next_2014 = new
NodeSLL_2511532014(6);
73     //cetak list asli
74     System.out.print("Senarai berantai awal: ");
75     printList_2511532014(head_2014);
76     //tambahkan node baru di depan
77     System.out.print("tambah 1 simpul di depan: ");
78     int data_2014 = 1;
79     head_2014 = insertAtFront_2511532014(head_2014,
data_2014);
80     //cetak update list
81     printList_2511532014(head_2014);
82     //tambahkan ndoe baru dibelakang
83     System.out.print("tambah 1 simpul di belakang: ");
84     int data2_2014 = 7;
85     head_2014 = insertAtEnd_2511532014(head_2014,
data2_2014);
86     //cetak update list
87     printList_2511532014(head_2014);
88     System.out.print("tambah 1 simpul ke data 4: ");
89     int data3_2014 = 4;
90     int pos_2014 = 4;
91     head_2014 = insertPos_2511532014(head_2014, pos_2014,
data3_2014);

```



```

92         //cetak update list
93         printList_2511532014(head_2014);
94     }
95
96 }
97

```

2.2.3 Output

```

<terminated> TambahSLL_2511532014 [Java Application] C:\Users\asus\.p2
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

```

2.2.4 Analisis

Operasi insertAtFront memiliki kompleksitas waktu $O(1)$ karena tidak memerlukan penelusuran. Sebaliknya, insertAtEnd memerlukan waktu $O(n)$ karena program harus mengunjungi setiap simpul dari depan hingga akhir. Pada insertPos, ketelitian sangat diperlukan agar simpul yang ada di posisi target tidak "terputus" dari rantai saat simpul baru disisipkan di antaranya.

2.3 Class HapusSLL_2511532014

2.3.1 Langkah Pengerjaan

1. Implementasi deleteHead dengan memindahkan referensi head ke simpul berikutnya.
2. Implementasi removeLastNode dengan mencari simpul kedua dari belakang (second last) dan mengubah next-nya menjadi null.
3. Implementasi deleteNode pada posisi tertentu dengan menyambungkan simpul prev langsung ke simpul setelah simpul yang dihapus.

2.3.2 Kode Program

```

1 package pekan5_2511532014;
2
3 public class HapusSLL_2511532014 {
4     public static NodeSLL_2511532014
5     deleteHead_2511532014(NodeSLL_2511532014 head_2014) {
6         //jika SLL kosong
7         if (head_2014 == null) {
8             return null;
9         }
10    }

```

```

9          //pindahkan head ke node berikutnya
10         head_2014 = head_2014.next_2014;
11         return head_2014;
12     }
13     // fungsi menghapus node terakhir SLL
14     public static NodeSLL_2511532014
removeLastNode_2511532014(NodeSLL_2511532014 head_2014) {
15         //jika list kosong return null
16         if (head_2014 == null) {
17             return null;
18         }
19         //jika list satu node, hapus node return null
20         if (head_2014.next_2014 == null) {
21             return null;
22         }
23         //temukan ndoe terakhir kedua
24         NodeSLL_2511532014 secondLast_2014 = head_2014;
25         while (secondLast_2014.next_2014.next_2014 != null) {
26             secondLast_2014 = secondLast_2014.next_2014;
27         }
28         //hapus node terakhir
29         secondLast_2014.next_2014 = null;
30         return head_2014;
31     }
32     //fungsi menghapus node di posisi tertentu
33     public static NodeSLL_2511532014
deleteNode_2511532014(NodeSLL_2511532014 head_2014, int
position_2014) {
34         NodeSLL_2511532014 temp_2014 = head_2014;
35         NodeSLL_2511532014 prev_2014 = null;
36         //jika linked list null
37         if (temp_2014 == null) {
38             return head_2014;
39         }
40         //kasus 1: head dihapus
41         if (position_2014 == 1) {
42             head_2014 = temp_2014.next_2014;
43             return head_2014;
44         }
45         //kasus 2: menghapus ndoe ditengah
46         //telusuri ke node yg dihapus
47         for (int i = 1; temp_2014 != null && i < position_2014;
i++) {
48             prev_2014 = temp_2014;
49             temp_2014 = temp_2014.next_2014;
50         }
51         //jika ditemukan, hapus node
52         if (temp_2014 != null) {
53             prev_2014.next_2014 = temp_2014.next_2014;
54         } else {
55             System.out.println("data tidak ada");
56         }
57         return head_2014;
58     }
59     // fungsi mencetak sll
60     public static void printList_2511532014(NodeSLL_2511532014
head_2014) {

```

```

61     NodeSLL_2511532014 curr_2014 = head_2014;
62     while (curr_2014.next_2014 != null) {
63         System.out.print(curr_2014.data_2014 + "-->");
64         curr_2014 = curr_2014.next_2014;
65     }
66     if (curr_2014.next_2014 == null) {
67         System.out.print(curr_2014.data_2014);
68     }
69     System.out.println();
70 }
71 //kelas main
72 public static void main(String[] args) {
73     //buat sll 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
74     NodeSLL_2511532014 head_2014 = new NodeSLL_2511532014(1);
75     head_2014.next_2014 = new NodeSLL_2511532014(2);
76     head_2014.next_2014.next_2014 = new
NodeSLL_2511532014(3);
77     head_2014.next_2014.next_2014.next_2014 = new
NodeSLL_2511532014(4);
78     head_2014.next_2014.next_2014.next_2014.next_2014 = new
NodeSLL_2511532014(5);
79     head_2014.next_2014.next_2014.next_2014.next_2014.next_2014 =
new NodeSLL_2511532014(6);
80     //cetak list awal
81     System.out.println("list awal: ");
82     printList_2511532014(head_2014);
83     //hapus head
84     head_2014 = deleteHead_2511532014(head_2014);
85     System.out.println("List setelah head dihapus: ");
86     printList_2511532014(head_2014);
87     //hapus node terakhir
88     head_2014 = removeLastNode_2511532014(head_2014);
89     System.out.println("list setelah simpul terakhir di
hapus: ");
90     printList_2511532014(head_2014);
91     //deleting node at position 2
92     int position_2014 = 2;
93     head_2014 = deleteNode_2511532014(head_2014,
position_2014);
94     //printlist after deletion
95     System.out.println("List setelah posisi 2 dihapus: ");
96     printList_2511532014(head_2014);
97 }
98
99 }
100

```

2.3.3 Output

```

<terminated> HapusSLL_2511532014 [Java Application] C:\Users\asus
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
list setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

2.3.4 Analisis

Penghapusan pada SLL dilakukan dengan memutus aliran *pointer*. Saat menghapus simpul tengah, simpul yang dihapus secara teknis masih ada di memori tetapi tidak lagi memiliki referensi yang menunjuk kepadanya, sehingga nantinya akan dibersihkan oleh *Garbage Collector* Java. Analisis pada `removeLastNode` menunjukkan bahwa kita harus berhenti satu simpul sebelum akhir agar bisa memutuskan hubungan ke simpul terakhir.

2.4 Class PencarianSLL_2511532014

2.4.1 Langkah Pengerjaan

1. Membuat metode traversal untuk mengunjungi dan mencetak data setiap simpul hingga null.
2. Membuat metode `searchKey` dengan logika perulangan `while` yang membandingkan data setiap simpul dengan kunci pencarian.

2.4.2 Kode Program

```

1 package pekan5_2511532014;
2
3 public class PencarianSLL_2511532014 {
4     static boolean searchKey_2511532014(NodeSLL_2511532014
5     head_2014, int key_2014) {
6         NodeSLL_2511532014 curr_2014 = head_2014;
7         while (curr_2014 != null) {
8             if (curr_2014.data_2014 == key_2014){
9                 return true;
10            }
11            curr_2014 = curr_2014.next_2014;
12        }
13        return false;
14    }
15    public static void traversal_2511532014(NodeSLL_2511532014
16    head_2014) {

```

```

15      //mulai dari head
16      NodeSLL_2511532014 curr_2014 = head_2014;
17      //telusuri sampai pointer null
18      while (curr_2014 != null) {
19          System.out.print(" " + curr_2014.data_2014);
20          curr_2014 = curr_2014.next_2014;
21      }
22      System.out.println();
23  }
24  public static void main(String[] args) {
25      NodeSLL_2511532014 head_2014 = new
NodeSLL_2511532014(14);
26      head_2014.next_2014 = new NodeSLL_2511532014(21);
27      head_2014.next_2014.next_2014 = new
NodeSLL_2511532014(13);
28      head_2014.next_2014.next_2014 = new
NodeSLL_2511532014(30);
29      head_2014.next_2014.next_2014.next_2014 = new
NodeSLL_2511532014(10);
30      System.out.print("Penelusuran SLL : ");
31      traversal_2511532014(head_2014);
32      //data yang akan dicari
33      int key_2014 = 30;
34      System.out.print("cari data " + key_2014 + " = ");
35      if (searchKey_2511532014(head_2014, key_2014)){
36          System.out.println("Ketemu");
37      } else {
38          System.out.println("Tidak ada");
39      }
40  }
41  }
42

```

2.4.3 Output

```

<terminated> PencarianSLL_2511532014 [Java
Penelusuran SLL : 14 21 30 10
cari data 30 = Ketemu

```

2.4.4 Analisis

Pencarian pada SLL bersifat linear ($O(n)$). Program harus memeriksa simpul satu per satu karena simpul pada Linked List tidak memiliki indeks langsung seperti array. Jika data ditemukan di tengah proses penelusuran, fungsi akan langsung mengembalikan nilai true dan menghentikan perulangan untuk efisiensi.

BAB III

KESIMPULAN

Berdasarkan praktikum pekan 5, dapat disimpulkan bahwa:

1. **Struktur Dinamis:** SLL memungkinkan pengelolaan data yang lebih fleksibel dalam penggunaan memori dibandingkan array statis.
2. **Ketergantungan Pointer:** Seluruh operasi pada SLL sangat bergantung pada manipulasi variabel referensi (next). Kesalahan dalam pengaturan *pointer* dapat menyebabkan data hilang atau terjadi *infinite loop*.
3. **Efisiensi Operasi:** Penambahan dan penghapusan di awal senarai sangat efisien ($O(1)$), namun operasi pada akhir atau posisi tertentu membutuhkan waktu penelusuran ($O(n)$).

DAFTAR PUSTAKA

Laporan Tugas

1. Class Pasien_2511532014

a. Langkah Pengerjaan

- i. Mendefinisikan atribut namaPasien_2014, penyakit_2014, dan nomorAntrian_2014 sebagai identitas data pasien.
- ii. Menambahkan atribut next_2014 bertipe objek itu sendiri sebagai referensi ke simpul pasien berikutnya.
- iii. Membuat konstruktor untuk menginisialisasi identitas pasien dan mengatur nilai next_2014 menjadi null secara default.
- iv. Menyediakan metode Getter dan Setter untuk setiap atribut guna menerapkan prinsip enkapsulasi data.

b. Kode Program

```
1. package pekan5_2511532014;
2.
3. public class Pasien_2511532014 {
4.     String namaPasien_2014;
5.     String penyakit_2014;
6.     int nomorAntrian_2014;
7.     Pasien_2511532014 next_2014;
8.
9.     // Konstruktor
10.    public Pasien_2511532014(String namaPasien_2014, String
    penyakit_2014, int nomorAntrian_2014){
11.        this.namaPasien_2014 = namaPasien_2014;
12.        this.penakit_2014 = penyakit_2014;
13.        this.nomorAntrian_2014 = nomorAntrian_2014;
14.        this.next_2014 = null;
15.    }
16.
17.    //getter
18.    public String getNamaPasien_2014() {
19.        return namaPasien_2014;
20.    }
21.    public String getPenyakit_2014() {
22.        return penyakit_2014;
23.    }
24.    public int getNomorAntrian_2014() {
25.        return nomorAntrian_2014;
26.    }
27.    public Pasien_2511532014 getNext_2014() {
28.        return next_2014;
29.    }
30.
31.    //setter
32.    public void setNamaPasien(String namaPasien_2014) {
33.        this.namaPasien_2014 = namaPasien_2014;
34.    }
35.    public void setPenyakit_2014(String penyakit_2014) {
36.        this.penakit_2014 = penyakit_2014;
37.    }
38.    public void setNomorAntrian_2014(int nomorAntrian_2014) {
```



```

39.         this.nomorAntrian_2014 = nomorAntrian_2014;
40.     }
41.     public void setNext_2014(Pasien_2511532014 next_2014) {
42.         this.next_2014 = next_2014;
43.     }
44.
45. }
46.

```

c. Analisis

Kelas Pasien_2511532014 berfungsi sebagai struktur simpul (*node*) khusus dalam senarai berantai tunggal yang menyimpan informasi heterogen. Analisis pada desain kelas ini menunjukkan bahwa objek ini tidak hanya menyimpan nilai primitif seperti nomor antrian, tetapi juga referensi memori (*next_2014*) yang memungkinkan terbentuknya rantai antrian dinamis di dalam memori. Penggunaan metode akses (*getter/setter*) memastikan bahwa perubahan status simpul dalam antrian dapat dikelola secara terkontrol oleh kelas pengelola.

2. Class RumahSakit_2511532014

a. Langkah Pengerjaan

- i. Mendeklarasikan variabel statis *head_2014* sebagai penanda awal antrian dan *counter_2014* untuk penomoran otomatis.
- ii. Mengimplementasikan metode *daftarPasien_2014* dengan logika Insert Tail, yaitu menelusuri senarai hingga akhir sebelum menambahkan simpul pasien baru.
- iii. Mengimplementasikan metode *panggilPasien_2014* dengan logika Delete Head, yaitu memproses pasien terdepan lalu menggeser referensi *head* ke simpul selanjutnya.
- iv. Menyusun metode *cariPasien_2014* yang melakukan perbandingan teks menggunakan *equalsIgnoreCase* untuk mencari nama pasien tertentu dalam senarai.
- v. Membuat metode *cekStatus_2014* untuk menghitung total elemen dalam senarai berantai secara waktu nyata.
- vi. Mengelola aliran interaksi pengguna melalui menu berbasis *do-while*.

b. Kode Program

```

1. package pekan5_2511532014;
2.
3. import java.util.Scanner;
4.
5. public class RumahSakit_2511532014 {
6.     static Pasien_2511532014 head_2014 = null;
7.     static int counter_2014 = 0;
8.     //menu

```

```

9.         public static void tampilkanMenu_2014() {
10.             System.out.println("\n=== Antrian Rumah Sakit
NIM:2511532014 ===");
11.             System.out.println("1. Daftarkan Pasien
(Insert)");
12.             System.out.println("2. Tombol Panggil Pasien
(Delete Head)");
13.             System.out.println("3. Tampilkan Antrian
(Display)");
14.             System.out.println("4. Cari Pasien (Search)");
15.             System.out.println("5. Cek Status Antrian");
16.             System.out.println("6. Keluar");
17.         }
18.
19.         //mendaftarkan pasien / insert tail
20.         public static void daftarPasien_2014(String nama_2014,
String penyakit_2014) {
21.             counter_2014++;
22.             Pasien_2511532014 pasienBaru_2014 = new
Pasien_2511532014(nama_2014, penyakit_2014, counter_2014);
23.
24.             if (head_2014 == null) {
25.                 head_2014 = pasienBaru_2014;
26.             } else {
27.                 Pasien_2511532014 temp_2014 = head_2014;
28.                 while (temp_2014.getNext_2014() != null) {
29.                     temp_2014 = temp_2014.getNext_2014();
30.                 }
31.                 temp_2014.setNext_2014(pasienBaru_2014);
32.             }
33.             System.out.println("Pasien berhasil ditambahkan!
Nomor Antrian: " + counter_2014);
34.         }
35.
36.         //panggil pasien / delete head
37.         public static void panggilPasien_2014() {
38.             if (head_2014 == null) {
39.                 System.out.println("Antrian kosong!");
40.             } else {
41.                 System.out.println("Memanggil pasien:");
42.                 System.out.println("Nama : " +
head_2014.getNamaPasien_2014());
43.                 System.out.println("Keluhan : " +
head_2014.getPenyakit_2014());
44.                 System.out.println("Nomor : " +
head_2014.getNomorAntrian_2014());
45.                 head_2014 = head_2014.getNext_2014();
46.             }
47.         }
48.
49.         //Tampilkan antrian / Display
50.         public static void tampilkanAntrian_2014() {
51.             if (head_2014 == null) {
52.                 System.out.println("Antrian kosong!");
53.             } else {
54.                 Pasien_2511532014 temp_2014 = head_2014;

```

```

55.         System.out.println("=== Daftar Antrian
===");
56.         while (temp_2014 != null) {
57.             System.out.println("Nama      : " +
temp_2014.getNamaPasien_2014());
58.             System.out.println("Keluhan : " +
temp_2014.getPenyakit_2014());
59.             System.out.println("Nomor    : " +
temp_2014.getNomorAntrian_2014());
60.             System.out.println("-----");
61.             temp_2014 = temp_2014.getNext_2014();
62.         }
63.     }
64. }
65.
66. //cari Pasien / Search
67. public static void cariPasien_2014(String nama_2014) {
68.     if (head_2014 == null) {
69.         System.out.println("Antrian kosong!");
70.         return;
71.     }
72.
73.     Pasien_2511532014 temp_2014 = head_2014;
74.     boolean ditemukan_2014 = false;
75.
76.     while (temp_2014 != null) {
77.         if
78.         (temp_2014.getNamaPasien_2014().equalsIgnoreCase(nama_2014)) {
79.             System.out.println("Pasien ditemukan:");
80.             System.out.println("Nama      : " +
temp_2014.getNamaPasien_2014());
81.             System.out.println("Keluhan : " +
temp_2014.getPenyakit_2014());
82.             System.out.println("Nomor    : " +
temp_2014.getNomorAntrian_2014());
83.             ditemukan_2014 = true;
84.             break;
85.         }
86.         temp_2014 = temp_2014.getNext_2014();
87.     }
88.     if (!ditemukan_2014) {
89.         System.out.println("Pasien tidak
ditemukan!");
90.     }
91. }
92.
93. // Cek Status
94. public static void cekStatus_2014() {
95.     if (head_2014 == null) {
96.         System.out.println("Antrian kosong!");
97.     } else {
98.         int jumlah_2014 = 0;
99.         Pasien_2511532014 temp_2014 = head_2014;
100.
101.         while (temp_2014 != null) {
102.             jumlah_2014++;

```

```

103.         temp_2014 = temp_2014.getNext_2014();
104.     }
105.
106.     System.out.println("Jumlah pasien: " +
    jumlah_2014);
107.     System.out.println("Pasien terdepan:");
108.     System.out.println("Nama      : " +
    head_2014.getNamaPasien_2014());
109.     System.out.println("Keluhan : " +
    head_2014.getPenyakit_2014());
110.     System.out.println("Nomor    : " +
    head_2014.getNomorAntrian_2014());
111. }
112. }
113.
114. // Main Menu
115. public static void main(String[] args) {
116.     Scanner input_2014 = new
    Scanner(System.in);
117.     int pilihan_2014;
118.
119.     do {
120.         tampilkanMenu_2014();
121.         System.out.print("Pilihan: ");
122.         pilihan_2014 = input_2014.nextInt();
123.         input_2014.nextLine();
124.
125.         switch (pilihan_2014) {
126.         case 1:
127.             System.out.print("Masukan Nama Pasien: ");
128.             String nama_2014 = input_2014.nextLine();
129.             System.out.print("Masukan Keluhan: ");
130.             String Keluhan_2014 =
    input_2014.nextLine();
131.             daftarPasien_2014(nama_2014, Keluhan_2014);
132.             break;
133.         case 2:
134.             panggilPasien_2014();
135.             break;
136.         case 3:
137.             tampilkanAntrian_2014();
138.             break;
139.         case 4:
140.             System.out.print("Masukan nama pasien yang
    dicari: ");
141.             String cari_2014 = input_2014.nextLine();
142.             cariPasien_2014(cari_2014);
143.             break;
144.         case 5:
145.             cekStatus_2014();
146.             break;
147.         case 6:
148.             System.out.println("Terima kasih!");
149.             break;
150.         default:
151.             System.out.println("Pilihan tidak valid");
152.     }

```

```

153.
154.             } while (pilihan_2014 != 6);
155.
156.             input_2014.close();
157.         }
158.
159.     }
160.

```

c. Output

```

<terminated> RumahSakit_2511532014 [Java Application] C:\Users\
=== Antrian Rumah Sakit NIM:2511532014 ===
1. Daftarkan Pasien (Insert)
2. Tombol Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukan Nama Pasien: Faiz
Masukan Keluhan: Sakit Mata
Pasien berhasil ditambahkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM:2511532014 ===
1. Daftarkan Pasien (Insert)
2. Tombol Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih!

```

d. Analisis

Program ini mensimulasikan sistem antrian layanan kesehatan dengan menerapkan logika **FIFO (First In First Out)** melalui manipulasi Single Linked List. Analisis pada metode panggilPasien_2014 menunjukkan efisiensi $O(1)$ karena penghapusan dilakukan tepat pada simpul head. Sebaliknya, metode daftarPasien_2014 memerlukan penelusuran linear $O(n)$ untuk mencapai ujung senarai. Keunggulan sistem ini dibandingkan antrian berbasis array adalah tidak adanya batasan

kapasitas pasien (selama memori tersedia) dan tidak perlunya pergeseran data elemen lain saat pasien terdepan selesai dilayani. Penggunaan variabel